WEEK 14

Polyglot Incident Response and Final Project Clinic

Find the authoritative fact, trace propagation, repair safely, and communicate what the evidence supports.

14

A second database must earn its operating cost

Polyglot persistence is workload specialization, not a tool collection.

BENEFIT

One store serves a distinct model, access, scale, or lifecycle need.

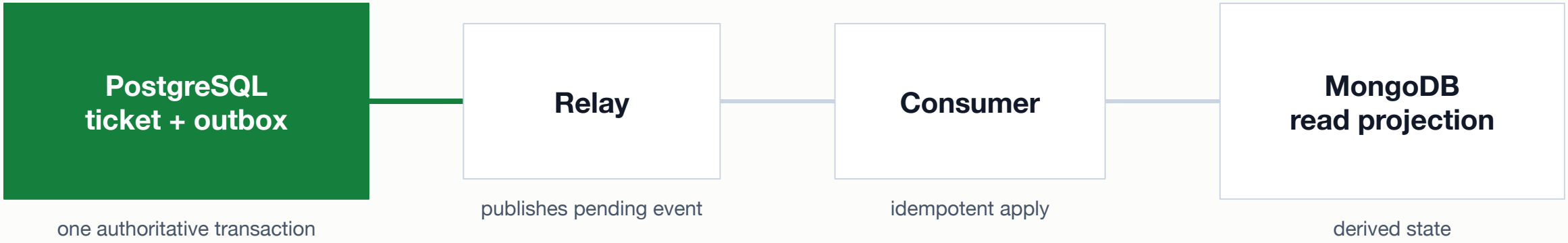
BOUNDARY

Every fact has one authoritative owner and a defined derived copy.

OBLIGATION

Propagation, access, monitoring, backup, recovery, and cost now cross systems.

A transactional outbox gives the change one commit boundary



The outbox closes the source update/event gap; delivery, retries, lag, and reconciliation still remain.

Delivery guarantees need stable identity and state transitions

Unsafe consumer behavior

- Applies every delivery as a new side effect
- Trusts arrival order without a source version
- Treats one success log as proof of convergence

Safer idempotent behavior

- Uses stable event and aggregate identifiers
- Applies only a newer source version
- Records progress and reconciles authoritative against derived state

Incident triage follows the fact across every boundary

Start with the user-visible disagreement, then locate the last confirmed state transition.

1. **Source**
   Did the authoritative transaction commit, with which version?
2. **Outbox**
   Was the stable event created and marked published?
3. **Consumer**
   Was it received, retried, rejected, or paused?
4. **Projection**
   Which source version and event ID are stored?
5. **Scope**
   Which other records or partition may share the failure?

The evidence packet locates one failed transition

Boundary	Observed evidence	What it supports	What it does not prove
PostgreSQL	status resolved, version 17	authoritative update committed	projection received it
Outbox	event 1008-17 published	event existed and relay marked publish	consumer applied it
Consumer	write timeout; retry worker paused	propagation stopped after receipt	only one event is affected
MongoDB	status open, version 16	projection is stale	manual edit is safest repair

Lab: Repair a stale MongoDB projection

Triage the evidence packet, preserve authoritative ownership, and write one workplace-readable incident report.

- Build the timeline, identify the owner, and state exactly where propagation stopped.
- Recommend an idempotent replay or resume action and five checks covering record, queue, duplicate effect, lag, and broader scope.
- Write a concise incident update with impact, evidence-based cause, mitigation, verification, limitation, and prevention.

SUBMIT ONE THING / one week_14_incident_report.md file

A safe repair restores the path instead of inventing truth

Repair the consumer, replay the stable event, and let the version rule reject duplicates.

MITIGATE

Restore scoped consumer access and contain further stale reads if impact requires it.

REPLAY

Resume from the recorded event or checkpoint through the normal idempotent handler.

VERIFY

Confirm version 17, queue health, no duplicate effect, normal lag, and affected partition.

DAY 2

Final evidence should survive a failed live demo

Prepared, redacted proof is part of the system, not a presentation shortcut.

Audit the project through one complete operating story

1. Model: one record's grain, identity, relationship, and integrity rule are visible.
2. Workload: one meaningful read and write have expected and observed evidence.
3. Protection: one unauthorized or invalid action is refused under the intended identity or rule.
4. Operations: one performance or recovery claim includes mechanism, verification, and limitation.

Final project clinic: Finish the highest-risk evidence gap

Use the canonical final-project page and rubric; do not create a separate Week 14 project submission.

- Audit model, workload, access, performance, recovery, and reproducibility evidence against the canonical rubric.
- Choose the one missing or weak claim that creates the greatest grading or demonstration risk.
- Complete and redact that evidence inside the existing project package, then prepare a non-live fallback.

SUBMIT ONE THING / no separate Week 14 artifact; update the existing final-project package

Cross-database reliability begins with ownership and traceable events

One fact has one owner; every derived copy needs propagation, reconciliation, and recovery rules.

- Which system owns the disputed fact?
- Where is the last confirmed transition in the timeline?
- Which verification proves the repair beyond one visible record?